Active Hierarchical Imitation and Reinforcement Learning

Yaru Niu
School of Electrical and Computer Engineering
Georgia Institute of Technology
yaruniu@gatech.edu

Yijun Gu
School of Computer Science
Georgia Institute of Technology
yjgu@gatech.edu

***Abstract*—Humans can leverage hierarchical structures to split a task into sub-tasks and solve problems efficiently. Both imitation and reinforcement learning or a combination of them with hierarchical structures have been proven to be an efficient way for robots to learn complex tasks with sparse rewards. However, in the previous work of hierarchical imitation and reinforcement learning, the tested environments are in relatively simple 2D games, and the action spaces are discrete. Furthermore, many imitation learning works focusing on improving the policies learned from the expert polices that are hard-coded or trained by reinforcement learning algorithms, rather than human experts. In the scenarios of human-robot interaction, humans can be required to provide demonstrations to teach the robot, so it is crucial to improve the learning efficiency to reduce expert efforts, and know human's perception about the learning/training process. In this project, we explored different imitation learning algorithms and designed active learning algorithms upon the hierarchical imitation and reinforcement learning framework we have developed. We performed an experiment where five participants were asked to guide a randomly initialized agent to a random goal in a maze. Our experimental results showed that using DAgger and reward-based active learning method can achieve better performance while saving more human efforts physically and mentally during the training process.**

***Index Terms*—Imitation Learning, Hierarchical Learning, Active Learning, Human-Robot Interaction**

I. Introduction

Hierarchical structures can help human to split a task into sub-tasks and solve problems efficiently. Both imitation and reinforcement learning or a combination of them with hierarchical structures have been proven to be an efficient way for robots to learn complex tasks [13], [14].

Imitation Learning (IL) is one of the efficient algorithms we use with an oracle or human demonstrators with (near) optimal performance. Sun and Bagnell compared the theoretical bound of performance using Reinforcement Learning (RL) with the bound using IL under some simple cases, and showed IL outperforms RL for a polynomial factor in general Markov Decision Process (MDP) [23]. Hierarchical Reinforcement Learning (HRL) is another efficient algorithm which uses temporal abstractions to help the agent to explore in more semantically meaningful action space, and thus improve the sample efficiency of RL algorithm [18]. Does combining two approaches further help us in complex tasks? In Hierarchical Imitation and Reinforcement Learning (HIRL) [13], the authors combined both methods in discrete state-action space, together with some strategies that restrict the place where learning occurs. They showed that their approach could decrease the expert's cost in training the agent, therefore, outperform other hierarchical approaches. Therefore, we developed a similar approach in this work as in HIRL: training high-level controller with IL and low-level controller with RL. Our task was built on a maze-navigation domain in continuous state and action space where we did not directly apply HIRL.

In the continuous setting, we seek to learn a set of policies together that is parameterized by the goal, and the problem is often referred to as contextual policy learning or multi-task learning. Some typical examples are locomotion task and robot arm manipulation task like [1], [20]. In our task, both the high-level controller and low-level controller are parameterized by their goal. Training such a policy is time-consuming. The model training in [20] requires several days of computation. As mentioned above, we expect that the agent can learn fast if it combines IL and uses a hierarchical structure. To further increase sample efficiency in this setting, we develop active learning methods and compare them with directly employing imitation learning on the high-level controller.

Furthermore, to evaluate how our proposed methods perform in a real human-robot interaction scenario and investigate how human feel about the learning/training process, we performed an experiment where five participants were asked to guide a randomly initialized agent to a random target in a maze. The results showed that using our hierarchical imitation and reinforcement learning framework with reward-based active learning method can achieve better performance than others while saving more human efforts physically and mentally during the training process.

II. Related Work

A. Interactive Imitation learning

Sequential prediction problems arise commonly in practice, and interactive imitation learning incorporates the learning processes in the teaching phase. Ross and Bagnell proposed an iterative meta-algorithm DAgger [21], which trains a stationary deterministic policy, that can be seen as a no-regret algorithm in an online learning setting. The approach is guaranteed to perform well under its induced distribution of states. However, in many cases the learner can only get access to a sub-optimal expert policy. Sun et al. proposed an interactive imitation learning algorithm, AggreVaTeD, on sequential prediction tasks [23]. AggreVaTeD can reach expert-level and even super-expert performance. We integrated the interactive imitation learning algorithms in our hierarchical framework to enable the human to teach the agent high-level policy.

B. Hierarchical Reinforcement Learning

Building agents that can learn hierarchical policies is a long-standing problem in Reinforcement Learning, for example, decompose MDP into smaller MDPs [6] or feudal RL that introduces spacial or temporal abstraction into the learning process [5]. Some theoretical results are given, where we can adopt options into MDPs and then obtain a semi-Markov decision process for high-level policies so that the agent can learn faster [24]. Recently, there are many new pieces of research that incorporates deep neural networks with hierarchical reinforcement learning. There are several automated HRL techniques that can work in discrete domains with a manually designed intrinsic reward for low-level policy [12]. Further works like Option-Critic architecture [2] and FeUdal Network [26] address the problem of predefined subgoals as well as extend HRL to continuous space. The main problem of HRL is that high-level policy learning is unstable due to the fact that low-level policy is changing. To address this issue, we use Hindsight Experience Replay [1] which was proposed in Hierarchical Actor-Critic (HAC) [14] or posterior probability optimization of the subgoal given low-level trajectories [17]. While most of the HRL methods only use a two-layer structure, HAC extends the framework to an arbitrary number of layers and each layer can be trained in parallel. It is further shown that HRL can be regarded as a multi-agent RL task and the non-stationary problem can be addressed in this view [11].

C. Active Learning

Active Learning is a popular approach for classification in the semi-supervised learning setting. The expert provides labels only when the agent asks for labels. The approach aims at saving the expert's labeling cost and it is very similar to our goal. In the context of LfD, researchers have proposed active learning methods based on confidence calculation [4], [15] or even learn how to active learn [3]. Despite the fact that it is a popular LfD method, we can only find a few pieces of research on the combination of multi-task learning and active learning [7], [22]. The method we adopted in our task is inspired by the work [22], where the author uses active learning for navigation tasks and the agent is selecting tasks (starting and ending points) for the human demonstrator. We also adopted the method by Hafner [8], where they use noise contrastive priors to estimate the reliable uncertainty of the neural networks.

D. Combining RL and LfD

The idea of combining IL and RL is not new [10], [19]. The proposed method Deep Q-Learning from Demonstration (DQfD) is taking IL as a "pre-training" step by pre-populating the replay buffer with demonstrations [10]. The pre-populated training examples can be used to warm start the agent. In HIRL, the combination of IL and RL is in the form of interaction between meta controller and low-level controller instead of warm start the agent. However, previous works only focused on the application in discrete state and action spaces [10], [13], [19]. In contrast, we built our pre-training IL and HRL in continuous tasks with continuous feedback.

III. Methods

A. Interactive Imitation Learning

In our method, we tried to impelement both DAgger and AggreVaTeD as our interactive imitation learning methods. We found that DAgger can fit in our hierarchical imitation and reinforcement learning framework well and learn the high-level policy quickly within reasonable number of interactive training episodes, while AggreVaTeD cannot improve the policy with the same scale of episodes. We thought that AggreVaTeD fails in our setting because AggreVaTeD performs policy gradient to update the learner's policy using the reward information in the environment, rather than directly learn from expert demonstrations like DAgger. In our settings, the rewards from the environment are sparse, and human can only afford reasonable number of interactive training time, so the policy learned from AggreVaTeD is very likely to gain nothing during this period. In this case, we finally used DAgger as our high-level policy learning method for the agent, and the following discussions are all based on DAgger.

B. Hierarchical Imitation and Reinforcement Learning

Here we introduce the Hierarchical Imitation Reinforcement Learning (HIRL) framework that can effectively learn multi-level policies in continuous spaces. We constructed the environment with a two-level hierarchy; the high level corresponds to choosing subtasks to reach the final goal and the low level corresponds to executing subtasks. We trained our high-level meta-controller using DAgger and low-level controller using a revised Deep Deterministic Policy Gradient (DDPG) method [16]. The algorithm of HIRL is shown in Alg. 1.

C. Active Learning

To improve the learning performance and efficiency of HIRL, and make it use-friendly during human-robot interaction in a goal-achieving task, we proposed the following active learning methods.

Algorithm 1 Hierarchical Imitation and Reinforcement Learning (High-Level)

Input: Environment, and pretrained low-level policy
Output: Learned high-level policy

Randomly initialize high-level policy π_{HI}
Initialize dataset $\mathcal{D} \leftarrow \emptyset$
Initialize mixing factor $\beta \leftarrow 1.0$
repeat
  Sample end goal g and initial state s_0 from environment
  repeat
    Get $(s, \pi^*(s))$ from demonstrator
    Generate subgoal $g_{\text{LO}} \leftarrow \beta\pi_{\text{HI}} + (1-\beta)\pi^*$
    $\mathcal{D} \leftarrow \mathcal{D} \bigcup \{(s, a)\}$
    Run low-level policy for fix horizon H
    if Reach update episodes **then**
      repeat
        $\pi_{\text{HI}} \leftarrow \pi_{\text{HI}} - \nabla\ell(s, a)$
      until number of update
      $\beta \leftarrow \beta/d$
    end if
    Execute low-level policy for horizon H given g_{LO}
  until Goal reached or maximum number of action taken
until 100 episodes

Algorithm 2 Noise-Based Active Learning

Input: Environment, candidate pool size, and number of different noises, current high-level policy π_θ, end goal g
Output: Initialized position s_g^*

Initialize noise buffer $\mathcal{N} \leftarrow \emptyset$
Initialize state dataset $\mathcal{S} \leftarrow \emptyset$
repeat
  Randomize noise n
  $\mathcal{N} \leftarrow \mathcal{N} \bigcup \{n\}$
until number of different noises
repeat
  Randomly choose a position s_g from environment
  repeat
    Add noise from $\mathcal{N}$ to s_o
    Get new state s
    $\mathcal{S} \leftarrow \mathcal{S} \bigcup \{s\}$
  until number of different noises
  Compute the variance with S, g and π_θ based on equation (1)
  Store variance
  $\mathcal{S} \leftarrow \emptyset$
until candidate pool size
Choose s_g^* based on equation (1)

1) Noise-Based: In a goal-achieving task, the agent state and goal state might not have the same dimension, and we assume that the agent state dimension includes the goal state dimension. And the noise-based method is based on the assumption that adding noise to the sub-dimension of the agent state should not change the expect of the optimal action, or it can be meaningless to calculate the variance of the output.

Let s represent the full dimension state of the agent state, s_g the sub-dimension of the agent state which is consistent with the goal state. First, we randomly choose a number of s_g as the candidates of the initialized sub-state. Then we add random uniform noise to other sub-dimension states s_o of the agent state rather than s_g. Then we get n agent state regarding the same s_g. Then we choose the s_g based on the equation written as

$$s_g^* = \arg\max_{s_g \in S_g} \sum_{i=1}^{n} [\pi_\theta(s_i(s_g)) - \bar{\pi}_{\theta_i}(s_i(s_g))]^2 \tag{1}$$

where S_g is the candidate pool of the s_g, and $s_i(\cdot)$ is the function to get the state after adding an random uniform noise to s_o, and π_θ is the current policy. In our task, s_g is the cartesian position of the agent. The algorithm is shown as Alg. 2.

2) Multi-Policy: Another AL method we have employed is similar to bagging in ensemble learning. We trained n policies with the same training dataset and tested how much these policies agree given a randomly selected state. This method can be formally written as

$$s_g^* = \arg\max_{s_g \in S_g} \sum_{i=1}^{n} [\pi_{\theta_i}(s(s_g)) - \bar{\pi}_{\theta_i}(s(s_g))]^2 \tag{2}$$

where S_g is the candidate pool of the s_g, and $s(\cdot)$ is a fixed mapping function from s_g to , and π_{theta_i} is train i policy. The algorithm is shown in Alg. 3.

Algorithm 3 Multi-Policy Active Learning

Input: Environment, candidate pool size, and number of different noises, current high-level policy bag $\{\pi_{\theta_i}\}_{i=1}^n$, end goal g
Output: Initialized position s_g^*

repeat
  Randomly choose a position s_g from environment
  Get new state s
  Compute the variance with s, g and $\{\pi_{\theta_i}\}_{i=1}^n$ based on equation (2)
  Store variance
until candidate pool size
Choose s_g^* based on equation (2)

3) Reward-Based: Different from the previous two methods that actively initialize the position (sub-state) of the agent at the start of each episode, reward-based active learning utilize the reward information from the environment and choose more useful data to store in the experience replay buffer. We assume the agent will only get the reward when it reaches the final goal, and each episode will end when the agent reach the final goal or the time step achieves its maximum. Within each episode, if the agent reaches the final goal, the data aggregation will follow the same mechanism as described in Alg. 1. If the agent cannot reach the goal, the data in $\mathcal{D}$ will be popped

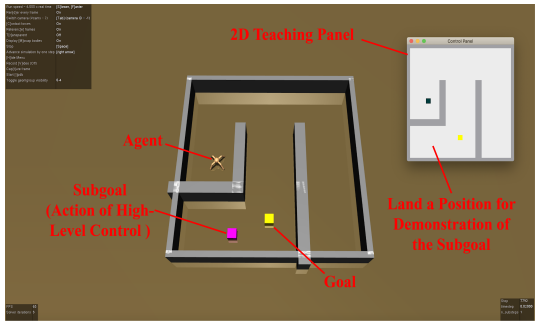

Fig. 1. The task overview of our experiment.

from the experience replay buffer following a probability:

$$p = e^{-t_d} \tag{3}$$

where t_d is a time step distance to the end of the episode. The algorithm is shown in Alg. 4.

Algorithm 4 Reward-Based Active Learning

Intialize $t \leftarrow 0$
for each episode **do**
  $t \leftarrow t + 1$
  Sample end goal g and initial state s_0 from environment
  repeat
    Get $(s^t, \pi^*(s^t))$ from demonstrator
    Generate subgoal $g_{\text{LO}} \leftarrow \beta\pi_{\text{HI}} + (1-\beta)\pi^*$
    $\mathcal{D} \leftarrow \mathcal{D} \bigcup \{(s^t, a^t)\}$
    Run low-level policy for fix horizon H
    Update high-level policy: $\pi_{\text{HI}} \leftarrow \pi_{\text{HI}} - \nabla\ell(s^t, a^t)$
    Update β: $\beta \leftarrow \beta/d$
    Execute low-level policy for horizon H given g_{LO}
  until Goal reached or maximum number of action taken
  if Goal is nor reached **then**
    for $t_d = 0$ to length of the previous episode **do**
      With probability e^{-t_d}, $\mathcal{D}$ pops (s^{t-t_d}, a^{t-t_d})
    end for
  end if
end for

IV. Human User Study

We evaluated our HIRL framework using Multi-Policy active learning (*DAgger+Multi-Policy AL*), and using Reward-Based active learning (*DAgger+Reward-Based AL*), compared to its vanilla version (*DAgger*). We conducted a within-subjects user study with five participants. As Noise-based active learning method has similar results with Multi-Policy one, we only evaluated the latter one in the experiment.

A. Task Environment

We built our experiment on a 3D maze navigation domain developed in MuJoCo [25] as shown in Fig. 1. In each episode, the agent (standard Rllab Ant with a gear range of (-30, 30)) needs to navigate from one corner of the maze with randomly generated layouts to a target marked in yellow with a time limit of 500 steps. Grey rectangles are obstacles (wall), which the agent needs to avoid for survival. The contextual in-formation the agent receives is the pixel representation of a bird's-eye view of the environment, including the positions of the agent and the final goal.

B. Experimental Procedure

We recruited 5 able-bodied human participants (3 females and 2 males). We recruited participants to meet the following criteria: 8 years of age or older; able-bodied; no cognitive or visual impairments; fluent in spoken and written English; and not diagnosed with epilepsy. Participant ages ranged from 23 to 25 years old. For each participant, we conducted a total of 3 trials for the three algorithms: *DAgger*, *DAgger+Multi Policy AL*, and *DAgger+Reward-based AL*. Each trial consists of 100 episodes. We randomized the ordering of the three methods for each participant according to randomized counter balancing design.

During the study, the participants moved the mouse the provide demonstrations of the subgoal positions to help agents to navigate to the target. The participants were instructed to show a subgoal position towards the target but also make sure not to be too far from the current position of the agent. Prior to the study, we gave participants an unscripted practice trial with 30 episodes to familiarize themselves on interacting with the agent and how to provide robust demonstrations to accomplish the tasks.

C. Objective Measures

We used the following measures as objective measures of performance:

- **Success rate:** average task success percentages over 100 test episodes; task success is defined by the agent reaching the final goal.
- **Expert cost:** average number of demonstrations in one episode over 100 test episodes.
- **Attempts:** average number of subgoals generated to reach the final goal in one episode over 100 test episodes.

D. Subjective Measures

Seeking to capture the participants' perceptions of the policy learning abilities, we used the questionnaire with three statements pertaining to perceptions of the task physical demand, performance, and human effort. For each statement, participants were asked to rate how much they agreed with the statement on the interval scale from 1 to 5. We based this 5-Point Likert Scale questionnaire on NASA-TLX [9].
The four statements follow:

- How physically demanding was the task? (1 = Very low, 5 = Very high)
- How successful were you in accomplishing what you were asked to do? (1 = Failure, 5 = Perfect)
- How successful were you in accomplishing what you were asked to do? (1 = Very low, 5 = Very high)

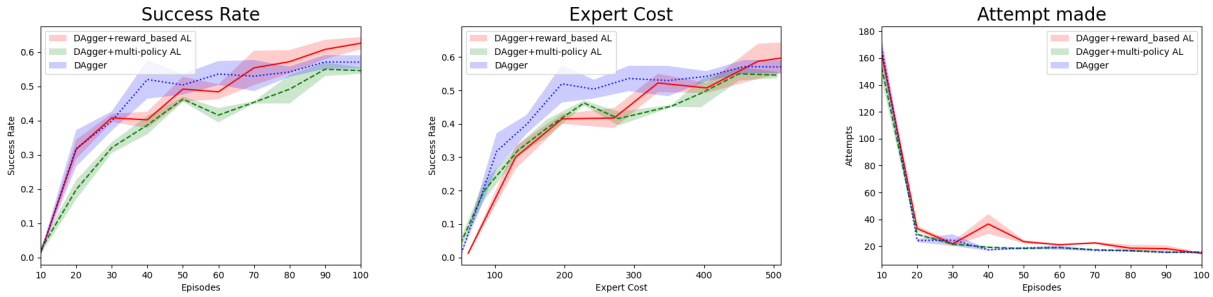

Fig. 2. Experiment Results over 300 test episodes. (a) Success rate per episode. (b) Success rate versus the expert cost. (c) Attempts per episode.

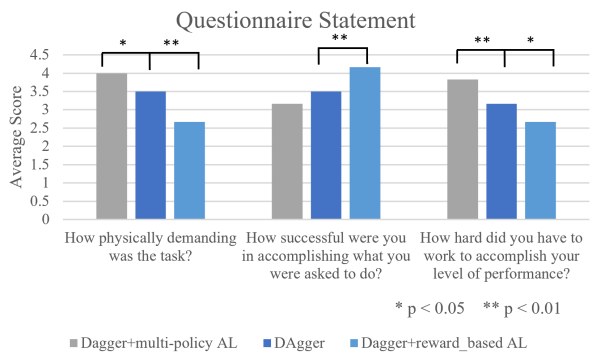

Fig. 3. Comparison of 5-point scale questionnaire responses for *DAgger*, *DAgger+Multi-policy AL*, and *DAgger+Reward-based AL*. Responses are averaged over all participants. p-values are computed with one-way ANOVA test.

V. Results

A. Objective Measures

We compared our AL-based HIRL algorithms with the vanilla version. Fig. 2 displays the results of the comparison.

1) Success rate: Fig. 2 (left) shows the success rate of three algorithms over 100 episodes. *DAgger+Reward-based AL* outperforms *DAgger* and achieves consistently highest success rate among the three algorithms after about 60 episodes. It exceeds 60% in fewer than 100 episodes. *DAgger+Multi Policy AL* performs worse than *DAgger* and has largest variation among the three algorithms. The figure displays the median as well as the range from minimum to maximum success rate over 5 random executions of the algorithms.

2) Expert cost: Fig. 2 (middle) shows the same success rate, but as a function of the expert cost. *DAgger* employs less expert demonstrations compared to other two algorithms when achieving a success rate lower than 50%. To achieve a success rate more than 50%, *DAgger+Reward-based AL* achieves most savings in expert cost. *DAgger+Multi Policy AL* needs the most expert costs within 100 episodes.

3) Attempts: Fig. 2 (right) shows that all of the three algorithms decrease suddenly when trained after the first 20 episodes and have a slow decrease after 20 episodes. There is no obvious difference among them.

Through the results, we concluded that *DAgger+Reward-based AL* performs better than the flat version (*DAgger*) while *DAgger+Multi-policy AL* performs worse than *DAgger*.

B. Subjective Measures

Fig. 3 displays a comparison of questionnaire responses between AL-based HIRL algorithms and vernilla version when responses are averaged over all participants for each questionnaire statement. To determine the statistical difference between algorithms, we applied a one-way ANOVA test. The computed p-values are depicted in Fig. 3.

- ***DAgger* and *DAgger+Multi-Policy AL*:** Participants provided higher responses related to physical demand and effort and lower response related to success in which robot used *DAgger+Multi-Policy AL* algorithm to learn from demonstrations. From Fig. 3, we observed a statistically significant difference at the $p < .05$ level for questions relating to physical demand, and a less significant difference at the $p < .001$ level for effort question between *DAgger* and *DAgger+Multi-Policy AL*.
- ***DAgger* and *DAgger+Reward-based AL*:** Participants provided lower responses related to physical demand and effort and higher response related to success in which robot used *DAgger+Reward-based AL* algorithm to learn from demonstrations. From Fig. 3, we observed a statistically significant difference at the $p < .001$ level for questions relating to physical demand, task success and a less significant difference at the $p < .05$ level for effort between *DAgger* and *DAgger+Reward-based AL*.

These results indicate that participants perceived a noticeable improvement in an agent's performance and easiness when using *DAgger+Reward-based AL* to provide guided demonstrations while perceive a decrease when using *DAgger+Multi-Policy AL*.

VI. Conclusion and Future Work

We explored active learning algorithms with hierarchical imitation and reinforcement learning framework. We presented two active learning-based HIRL algorithms: *DAgger+Multi-policy AL* and *DAgger+Reward-based AL*. We also provided evidence that *DAgger+Reward-based AL* can improve the performance and reduce the cost of expert feedback in hierarchical imitation and reinforcement learning. Overall, our results suggest that active reward learning can be used to help speed up learning from demonstration feedback and can be extended in the future work.

ACKNOWLEDGMENT

This is a course project for CS 7633 Human Robot Interaction in Georgia Institute of Technology. We thank Prof. Sonia Chernova for feedback about this work and Zuoxing Tang for assistance.

REFERENCES

[1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, pages 5048–5058, 2017.

[2] Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.

[3] Kalesha Bullard, Yannick Schroecker, and Sonia Chernova. Active learning within constrained environments through imitation of an expert questioner. *arXiv preprint arXiv:1907.00921*, 2019.

[4] Sonia Chernova and Manuela Veloso. Interactive policy learning through confidence-based autonomy. *Journal of Artificial Intelligence Research*, 34:1–25, 2009.

[5] Peter Dayan and Geoffrey E Hinton. Feudal reinforcement learning. In *Advances in neural information processing systems*, pages 271–278, 1993.

[6] Thomas G Dietterich. Hierarchical reinforcement learning with the maxq value function decomposition. *Journal of artificial intelligence research*, 13:227–303, 2000.

[7] Alexander Fabisch and Jan Hendrik Metzen. Active contextual policy search. *The Journal of Machine Learning Research*, 15(1):3371–3399, 2014.

[8] Danijar Hafner, Dustin Tran, Alex Irpan, Timothy Lillicrap, and James Davidson. Reliable uncertainty estimates in deep neural networks using noise contrastive priors. *arXiv preprint arXiv:1807.09289*, 2018.

[9] Sandra G Hart. Nasa-task load index (nasa-tlx); 20 years later. In *Proceedings of the human factors and ergonomics society annual meeting*, volume 50, pages 904–908. Sage publications Sage CA: Los Angeles, CA, 2006.

[10] Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Ian Osband, et al. Deep q-learning from demonstrations. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.

[11] Abdul Rahman Kreidieh, Samyak Parajuli, Nathan Lichtle, Yiling You, Rayyan Nasr, and Alexandre M Bayen. Inter-level cooperation in hierarchical reinforcement learning. *arXiv preprint arXiv:1912.02368*, 2019.

[12] Tejas D Kulkarni, Karthik Narasimhan, Ardavan Saeedi, and Josh Tenenbaum. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In *Advances in neural information processing systems*, pages 3675–3683, 2016.

[13] Hoang M Le, Nan Jiang, Alekh Agarwal, Miroslav Dudík, Yisong Yue, and Hal Daumé III. Hierarchical imitation and reinforcement learning. *arXiv preprint arXiv:1803.00590*, 2018.

[14] Andrew Levy, George Konidaris, Robert Platt, and Kate Saenko. Learning multi-level hierarchies with hindsight. 2018.

[15] Mingkun Li and Ishwar K Sethi. Confidence-based active learning. *IEEE transactions on pattern analysis and machine intelligence*, 28(8):1251–1261, 2006.

[16] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[17] Ofir Nachum, Shixiang Shane Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems*, pages 3303–3313, 2018.

[18] Ofir Nachum, Haoran Tang, Xingyu Lu, Shixiang Gu, Honglak Lee, and Sergey Levine. Why does hierarchy (sometimes) work so well in reinforcement learning? *arXiv preprint arXiv:1909.10618*, 2019.

[19] Ashvin Nair, Bob McGrew, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Overcoming exploration in reinforcement learning with demonstrations. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6292–6299. IEEE, 2018.

[20] Xue Bin Peng, Glen Berseth, KangKang Yin, and Michiel Van De Panne. Deeploco: Dynamic locomotion skills using hierarchical deep reinforcement learning. *ACM Transactions on Graphics (TOG)*, 36(4):41, 2017.

[21] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635, 2011.

[22] David Silver, J Andrew Bagnell, and Anthony Stentz. Active learning from demonstration for robust autonomous navigation. In *2012 IEEE International Conference on Robotics and Automation*, pages 200–207. IEEE, 2012.

[23] Wen Sun, Arun Venkatraman, Geoffrey J Gordon, Byron Boots, and J Andrew Bagnell. Deeply aggrevated: Differentiable imitation learning for sequential prediction. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 3309–3318. JMLR. org, 2017.

[24] Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2):181–211, 1999.

[25] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.

[26] Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. Feudal networks for hierarchical reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 3540–3549. JMLR. org, 2017.